

LV15 포인터 변수의 원리와 문자열(2차원 배열)

<https://youtu.be/Y8hjb1fhFzI?feature=shared>

<https://youtu.be/CH1fl7NXy9E?feature=shared>

C++ 포인터 연산, 문자열 길이, 문자열 비교

포인터 변수 심화

포인터 연산의 기본 개념

포인터 변수는 배열처럼 사용이 가능합니다. 일반적으로는 (포인터 변수 + 숫자)를 이용해서 배열처럼 접근해야 하지만 해당 방법은 가독성도 떨어지고 사용하는 사람 입장에서 너무 불편하기 때문에 배열처럼 사용 가능한 기능이 들어가 있다.

포인터와 배열의 관계

```
#include <iostream>
using namespace std;

int main()
{
    char test[10] = "HELLO";

    char ch = 'A'; // 변수 ch의 값은 'A'이다.
    char* p = &ch; // 포인터 변수 p의 값은 변수 ch의 주소값이다.
    // int* n = &ch; // 오류! 자료형이 다름

    p = test; // test 배열의 첫번째 값을 p에 대입한다.
    // p = &test[0]; // 위와 동일한 의미

    // 포인터 연산을 통한 배열 접근
    cout << "*p: " << *p << endl; // H
    cout << "**(p+1): " << *(p + 1) << endl; // E
    cout << "**(p+2): " << *(p + 2) << endl; // L

    // 배열 표기법으로 접근 (더 편리함)
    cout << "p[0]: " << p[0] << endl; // H
    cout << "p[1]: " << p[1] << endl; // E
    cout << "p[2]: " << p[2] << endl; // L

    // 다른 배열을 가리키도록 변경
    int test2[10] = { 1, 2, 34, 5, 5, 6, 7, };
    int* pint = test2;

    cout << "test2: " << *pint << endl; // 1
    cout << "**(test2 + 1): " << *(pint + 1) << endl; // 2
    cout << "**(test2 + 2): " << *(pint + 2) << endl; // 34

    cout << "test2[0]: " << pint[0] << endl; // 1
    cout << "test2[1]: " << pint[1] << endl; // 2
```

```
cout << "test2[2]: " << pint[2] << endl; // 34

return 0;
}
```

포인터 산술 연산의 핵심 원리

중요한 공식:

- `arr[i] = (arr + i)`
- 배열의 이름 `arr` 은 배열의 시작 주소를 가리킵니다

메모리 구조 이해:



```
int arr[3] = {10, 20, 30};
```

주소	메모리	포인터 표현
0x10	← [10]	← arr, arr+0, &arr[0]
0x14	← [20]	← arr+1, &arr[1]
0x18	← [30]	← arr+2, &arr[2]

포인터 산술 연산의 특징:

- `arr + 1` 은 주소값에 1을 더하는 것이 아님
- 해당 자료형의 크기만큼 주소가 증가
- `int` 형이면 4바이트, `char` 형이면 1바이트씩 이동

배열과 포인터 활용 비교

다만 배열의 크기를 넘게 사용이 되기 때문에 사용시 해당 배열의 크기를 넘어가지 않도록 주의해야 한다. 따라서 배열의 이름과 포인터 사이에는 다음과 같은 공식이 성립합니다.

안전한 사용법:

```
int arr[5] = {1, 2, 3, 4, 5};
int* ptr = arr;

// 올바른 사용 (인덱스 0~4)
for(int i = 0; i < 5; i++)
{
    cout << ptr[i] << " "; // 안전
}
```

```

}

// 위험한 사용 (배열 범위 초과)
cout << ptr[10] << endl; // 위험! 메모리 침범

```

포인터 변수의 활용

문자열 길이 구하는 함수

문자열의 길이를 구하는 사용자 정의 함수를 만들어 보겠습니다. 문자열은 null 문자(`\0`)로 끝나므로 이를 이용하여 길이를 계산할 수 있습니다.

```

#include <iostream>
using namespace std;

// 문자열 길이를 구하는 함수
int strlen(char* str)
{
    int len = 0;
    for (size_t i = 0; i < 10; i++)
    {
        if (str[i] == '\0')
        {
            len = i;
            break;
        }
    }
    return len;
}

int main()
{
    char str[3][10] = { "ABC", "DABC", "ABCGH" };

    for (size_t j = 0; j < 3; j++)
    {
        int len = strlen(str[j]);
        cout << len << endl;
    }

    return 0;
}

```

함수 동작 원리:

1. **매개변수:** `char* str` - 문자열의 시작 주소를 받음
2. **반복문:** 각 문자를 하나씩 검사
3. **종료 조건:** null 문자(`\0`)를 만나면 반복 종료
4. **반환값:** null 문자가 나타난 위치 = 문자열의 길이

출력 결과:

```

3 // "ABC"의 길이
4 // "DABC"의 길이
5 // "ABCGH"의 길이

```

strlen 함수 구현

```
int strlen_safe(char* str, int max_size)
{
    int len = 0;
    for (int i = 0; i < max_size; i++)
    {
        if (str[i] == '\0')
        {
            return i; // 길이 반환
        }
    }
    return max_size; // null 문자를 찾지 못한 경우
}
```

개선 사항:

- **최대 크기 제한:** 무한 루프 방지
- **안전성 향상:** 배열 범위를 벗어나지 않음
- **예외 처리:** null 문자가 없는 경우 대응

문자열 비교

문자열 비교 함수 구현

두 문자열이 같은지 비교하는 함수를 직접 구현해 보겠습니다. C++에서 문자열은 문자 배열이므로 각 문자를 하나씩 비교해야 합니다.

```
#include <iostream>
using namespace std;

int main()
{
    char strA[256] = "HELLO";
    char strB[256] = "HELLO";

    // 배열의 이름은 해당 배열의 주소이기 때문에
    // 서로 다른 값을 비교할 수 있다.
    // 배열을 한 글자씩 순회하면서 비교해야 한다.

    int flag = 0;
    for (size_t i = 0; i < 256; i++)
    {
        if (strA[i] != strB[i])
        {
            flag = 1;
            break;
        }
    }

    if (flag == 1)
    {
        cout << "다르다" << endl;
    }
    else
    {
        cout << "같다" << endl;
    }
}
```

```
    return 0;
}
```

문자열 비교의 핵심 개념

왜 직접 비교가 안 되는가?

```
char strA[256] = "HELLO";
char strB[256] = "HELLO";

// 이것은 작동하지 않음!
if (strA == strB) // 주소 비교, 항상 false
{
    // 실행되지 않음
}
```

이유:

- `strA` 와 `strB` 는 서로 다른 메모리 주소를 가짐
- 배열의 이름은 해당 배열의 시작 주소를 나타냄
- 따라서 주소끼리 비교하게 되어 항상 다름

문자열 비교 함수

```
bool strcmp_custom(char* str1, char* str2)
{
    int i = 0;

    // 두 문자열을 동시에 순회
    while (str1[i] != '\0' && str2[i] != '\0')
    {
        if (str1[i] != str2[i])
        {
            return false; // 다름
        }
        i++;
    }

    // 길이가 다른 경우 체크
    if (str1[i] != str2[i])
    {
        return false; // 길이가 다름
    }

    return true; // 같음
}
```

알고리즘 설명:

1. **동시 순회:** 두 문자열을 같은 인덱스로 비교
2. **문자 비교:** 각 위치의 문자가 다르면 즉시 `false` 반환
3. **길이 검증:** 한 문자열이 먼저 끝나는지 확인
4. **결과 반환:** 모든 검사를 통과하면 `true` 반환

표준 라이브러리 활용

실무에서는 표준 함수 사용:

```
#include <cstring> // C 스타일 문자열 함수
#include <string> // C++ string 클래스

// C 스타일
if (strcmp(str1, str2) == 0)
{
    cout << "같음" << endl;
}

// C++ 스타일 (권장)
string str1 = "HELLO";
string str2 = "HELLO";
if (str1 == str2)
{
    cout << "같음" << endl;
}
```

장단점 비교:

방법	장점	단점
직접 구현	원리 이해, 세밀한 제어	코드 복잡, 버그 위험
C 함수	검증된 구현, 빠름	사용법 복잡, 안전성 낮음
C++ string	사용 편리, 안전함	메모리 오버헤드

실무 활용 팁

문자열 비교 최적화:

- 길이 먼저 비교: 길이가 다르면 즉시 false
- 대소문자 구분: 필요에 따라 `tolower()` 함수 활용
- 부분 문자열: `strncmp()` 함수로 일부만 비교

메모리 안전성:

- 항상 배열 크기 확인
- null 문자 존재 여부 검증
- 버퍼 오버플로우 방지

“강의는 많은데, 왜 나는 아직도 코드를 못 짤까?”

혼자 공부하다 보면 누구나 이런 고민을 하게 됩니다.

- 강의는 다 들었지만 막상 손이 안 움직이고,
- 복습을 하려 해도 무엇을 다시 봐야 할지 모르겠고,
- 질문할 곳도 없고,
- 유튜브는 결국 정답을 따라 치는 것밖에 안 되는 것 같고.

문제는 ‘연습’이 빠졌기 때문입니다.

단순히 강의를 듣는 것만으로는 실력이 늘지 않습니다.

실제 문제를 풀고, 고민하고, 직접 구현해보는 시간이 반드시 필요합니다.

그래서, 암암코딩 코칭은 다릅니다.

그냥 가르치지 않습니다.

스스로 설계하고, 코딩할 수 있게 만듭니다.

암암코딩 코칭에서는 단순한 예제가 아닌,

스스로 문제를 분석하고 구현해야 하는 연습문제를 제공합니다.

이 연습문제들은 다음과 같은 역량을 키우기 위해 설계되어 있습니다:

- 문제를 스스로 쪼개고 설계하는 힘
- 다양한 조건을 만족시키는 실제 구현 능력
- 기능 단위가 아닌, 프로그램 단위로 사고하는 습관
- 마침내 자신의 힘으로 코드를 끝까지 작성하는 경험

지금 필요한 건 더 많은 강의가 아닙니다.

코드를 스스로 완성해 나가는 훈련,

그것이 지금 실력을 끌어올릴 가장 현실적인 방법입니다.

👉 자세한 안내 보기: [프리미엄 코칭 안내 바로가기](#)

👉 또는 카카오톡 상담방: [얏얏코딩 상담방](#)

▼ oopy:hide

연습문제



문제 1번

아래와 같이 vect라는 배열에 하드코딩 해 주세요

3	5	1	1	2	3	2
---	---	---	---	---	---	---

4칸짜리 1차원 배열에 숫자 4개를 입력받아주세요

입력받은 각 숫자가 vect 배열 안에 몇개 존재하는지 출력 해 주세요

입력 예시

4 1 2 3

출력 예시

4=0개

1=2개

2=2개

3=2개





문제 2번

아래 배열을 하드코딩 해 주세요

10	50	40	20	30	40
----	----	----	----	----	----

이제 6칸짜리 새로운 배열을 하나 더 만들고 숫자 6개를 입력받으세요

그리고 각각의 숫자보다 큰 값이 몇개 있는지 Count해서 출력 해 주세요

만약 15 35 80 35 45 5를 입력받았다면

15	35	80	35	45	5
----	----	----	----	----	---

이렇게 배열에 값을 채우고, 하드코딩한 배열에 입력받은 값보다 큰 숫자가 몇개 존재하는지

출력 하시면 됩니다

입력 예시

15 35 80 35 45 5

출력 예시

15=5개

35=3개

80=0개

35=3개

45=1개

5=6개



문제 3번

두 문장을 입력받아주세요 (최대 5글자)

그리고 각 문장을 정렬 합니다

정렬한 문장을 한 문장으로 옮기고 출력 해 주세요

[힌트]

가장 먼저 각 문장이 몇 글자인지 알아야 합니다

예로들어 "DCBA", "QWLBB"을 입력받았다면

문장1 정렬 : ABCD

문장2 정렬 : BBLQW

한 문장으로 만들기 : ABCDBBLQW

입력 예시

DCBA

QWLBB

출력 예시

ABCDBBLQW



문제 4번

두 문장을 입력받아주세요 (최대 6글자)

두 문장이 완전히 같은 문장이라면 **"같음"**

다른문장이라면 **"다름"** 을 출력 해 주세요

[HINT] 두 문장이 모두 같은문장인지 파악하기 위해서는

일단 각 문장의 길이를 비교 해 봅니다.

조건1. 두 문장의 길이가 같은지

조건2. 다른 글자가 없는지

이 두 가지를 검사하면 됩니다.

입력 예시

MINS

MINST

출력 예시

다름

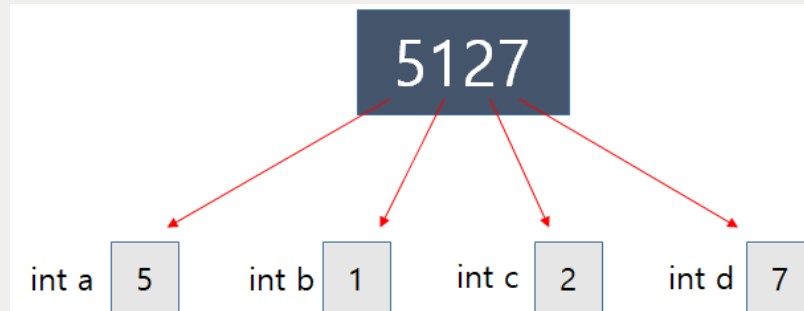


문제 5번

1000 ~ 9999 사이의 숫자 1개를 입력받아주세요

(문장으로 입력받지 않고 int형 변수에 입력 받습니다)

숫자를 쪼개서 각 자리 숫자를 4개의 변수에 옮겨담고, 각 자리를 따로 따로 출력 해 주세요



[HINT] MOD 연산자를 활용해보세요

/10, %10 등등 나눗셈과 MOD연산자를 잘 활용하면

각 자릿수를 쪼갤 수 있습니다

ex) t = 13

int a = t / 10; //앞자리

int b = t % 10; //뒷자리

입력 예시

5127

출력 예시

숫자5

숫자1

숫자2

숫자7



문제 6번

6명의 학생들이 일렬로 앉아있습니다.

앞사람과 뒷사람의 성적차이가 3보다 작게(<3) 차이가 나게끔 자리를 앉아야합니다

6명의 학생들 성적을 입력받고, 순차적으로 배열에 값을 넣어주세요

앞칸과 뒷칸의 성적차이가 3보다 작은지 검사하는 프로그램을 작성 해 주세요

ex) 예로들어 1 3 5 7 5 6 을 입력 받았다면

앞칸과 뒷칸이 3이상 차이나는 곳이 없으므로 "완벽한배치" 라고 출력 하세요

ex) 예로들어 2 1 6 4 3 2 를 입력 받았다면

1과 6의 차이가 3 이상 차이가 나기때문에 완벽한 배치가 아닙니다

이때는 "재배치필요" 라고 출력 해 주세요

입력 예시

1 3 5 7 5 6

출력 예시

완벽한배치



문제 7번

두 문장을 입력받으세요 (최대 10글자)

두 문장이 거꾸로 된 문장이 맞는지 확인하는 프로그램을 작성 해 주세요

ex) 예로들어 "ABCD" "DCBA" 를 입력받았다면

ABCD를 거꾸로 하면 DCBA가 맞습니다. 따라서 "거울문장" 이라고 출력 해 주세요

ex) 예로들어 "BKRQA" "ABC"를 입력받았다면

BKRQA를 거꾸로 하면 AQRKB 이기 때문에 "거울문장이아님" 이라고 출력 해 주세요

[힌트]

변수 t를 하나 더 만들어서 활용하면 됩니다.

t = length - 1 부터 시작해서

for문 돌때마다 1씩 빼주는 방식을 쓰면 됩니다.

입력 예시

NCQ

QCN

출력 예시

거울문장



문제 8번

네 문장을 2차배열에 입력받아주세요 (최대 6글자)

그리고 각 문장의 길이를 저장할 4칸짜리 1차원 배열을 만들어주세요

문장의 길이를 1차원 배열에 저장한 후 정렬하고 출력하시면 됩니다

ex) 만약 "ABCD", "BBQ", "UIOUIO", "MINS" 를 입력받았다면

A	B	C	D	\0		
B	B	Q	\0			
U	I	O	U	I	O	\0
M	I	N	S	\0		

이렇게 2차배열이 채워지고 각 문장의 길이를 1차원 배열에 저장 해 주세요

4	3	6	4
---	---	---	---

이 배열을 정렬하면 3 4 4 6 이 되므로 **3 4 4 6**을 출력 해 주시면 됩니다

입력 예시

ABCD

BBQ

UIOUIO

MINS

출력 예시

3 4 4 6



문제 9번

한 문장을 입력받으세요 (최대 10글자)

홀수번째 문자는 대문자 이어야 합니다

짝수번째 문자는 소문자 이어야 합니다

대소문자가 번갈아가면서 나오는 문장이 맞다면 "개구리문장"

그렇지 않다면 "일반문장" 을 출력 해 주세요

입력 예시

AgKeOqV

출력 예시

개구리문장



문제 10번

마을에 아래와 같은 알파벳 성을 가진 사람들이 살고 있습니다

(아래 1차원 배열을 하드코딩 해 주세요)

A	B	C	Z	E	T	Q
---	---	---	---	---	---	---

그리고 어느날, 범죄자 블랙리스트가 세상에 공개가 되었고

그 블랙리스트에 있는 사람이 마을 사람인지아닌지 확인 해 보려고합니다

블랙리스트는 다음과 같습니다

A	P	L	Q	F
---	---	---	---	---

블랙리스트 5명을 입력받아주세요

그리고 블랙리스트를 보면서 우리마을에 존재하는 사람인지 확인하는 프로그램을 작성 해 주세요

우리마을 사람이라면 "**마을사람**" 이라고 출력하면 됩니다

우리마을 사람이 아니라면 "**외부사람**" 이라고 출력하면 됩니다

입력 예시

APLQF

출력 예시

A=마을사람

P=외부사람

L=외부사람

Q=마을사람

F=외부사람



문제 11번

2차배열에 다섯문장을 입력받아주세요 (최대 10글자)

이 다섯 문장 중 **가장 긴 문장을 출력하시면 됩니다**

[HINT]

문장의 길이를 저장 할 int length[5] 배열을 만들어주세요

이 배열에 각 문장의 길이를 저장하고 MAX 값이 있는 index를 구하면

가장 긴 문장이 어디에 있는지 찾을 수 있어요

입력 예시

ABC

BBQ

KFC

SHOWSHOW

OPT

출력 예시

SHOWSHOW